CASE STUDY REPORT: EXPLORATION OF CLOUD SERVICE MODELS

A Practical Analysis of IaaS, PaaS, and SaaS Architectures Using AWS Free Tier Ecosystem

1. AIM AND OBJECTIVES

Aim

The aim of this case study is to conduct a detailed practical and architectural exploration of the three primary cloud service models—Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS)—by deploying and evaluating workloads within the Amazon Web Services (AWS) Free Tier platform.

Objectives

- Define and differentiate the core operational and architectural layers of IaaS, PaaS, and SaaS cloud delivery models.
- Deploy real-world representative workloads for each model on the AWS Free Tier platform: Amazon EC2 for IaaS, AWS Elastic Beanstalk for PaaS, and AWS Cloud9 / Google Workspace for SaaS.
- Evaluate control boundaries, administrative overhead, resource elasticity, security responsibilities, and cost structures across models.
- Establish an architectural decision framework to assist engineering teams in selecting the optimal cloud service model based on technical requirements and organizational maturity.

2. PROJECT OVERVIEW

Cloud computing has redefined modern software infrastructure by shifting enterprise computing from on-premise hardware to scalable, pay-as-you-go cloud services. Choosing the correct cloud service model impacts developer velocity, system maintainability, operational overhead, and infrastructure expenditure. This case study utilizes the **AWS Free Tier platform** to deploy, manage, and inspect real-world applications across **Infrastructure as a Service (IaaS)**, **Platform as a Service (PaaS)**, and **Software as a Service (SaaS)**, providing comparative technical insights into cloud resource management.

3. FUNDAMENTALS OF CLOUD SERVICE MODELS

Understanding the Shared Responsibility Model and abstraction boundaries between cloud service models is essential for cloud engineering decisions:

- **Infrastructure as a Service (IaaS):** Provides highly flexible, raw virtualized computing resources (virtual servers, storage, networking). Users retain full control over the operating system (OS), runtime environment, middleware, database configurations, and application security, while the provider manages physical hardware, hypervisors, and data center facilities.
- **Platform as a Service (PaaS):** Delivers a fully managed hardware and software framework designed for application development and deployment. The cloud provider automatically provisions servers, OS patching, runtime environments, and auto-scaling, allowing developers to focus strictly on writing source code and managing application data.
- **Software as a Service (SaaS):** Offers fully functioning end-user application software accessible via web browsers or API clients. The vendor completely manages the entire stack—from physical servers and networking to application code, databases, security updates, and storage. Users configure preferences and interact with the software directly.

4. FEATURE & PLATFORM COMPARISON FRAMEWORK

The deployed cloud service models are systematically evaluated across core technical and operational criteria in the comparison matrix below:

Evaluation Parameter	Infrastructure as a Service (IaaS)	Platform as a Service (PaaS)	Software as a Service (SaaS)
AWS Free Tier Representative Example	**Amazon EC2** (Elastic Compute Cloud t2.micro / t3.micro instance)	**AWS Elastic Beanstalk** (Managed Web App Platform) / AWS App Runner	**AWS Cloud9** (Cloud IDE) / **Amazon WorkDocs** / Google Workspace
User Control Level	**High:** Full administrative (root/sudo) access to OS, networking, security groups, and storage volumes.	**Moderate:** Control over application code, environment variables, configuration files, and deployment settings.	**Low:** Restricted to user account configuration, access rights, and custom workflow preferences.
Vendor Responsibility	Physical data centers, hardware infrastructure, virtualization layer (Hypervisor), and physical networking.	Hardware, hypervisor, OS provisioning, patch management, runtime maintenance, and auto-scaling logic.	Entire application stack including OS, runtime, database, app code, storage, backup, and security patches.
Deployment Complexity	**Complex:** Requires OS provisioning, SSH key management, firewall setup, dependency installation, and manual updates.	**Low to Moderate:** Streamlined deployment via code upload (ZIP, Git, Docker container) with automated provisioning.	**Minimal / Zero:** Instant access via web browser or client application with zero build requirements.
Scalability & Elasticity	Manual or custom Auto Scaling Groups (ASG) coupled with Elastic Load Balancers (ELB).	Native automated horizontal and vertical scaling based on CPU, memory, or HTTP traffic triggers.	Seamless cloud-native scaling completely abstracted and managed by the cloud vendor.
Primary Target Audience	System Administrators, DevOps Engineers, Cloud Infrastructure Architects.	Software Developers, Web Engineers, Rapid Prototyping Product Teams.	End-Users, Business Operators, Non-Technical Staff, Enterprise Personnel.

5. ARCHITECTURE AND INTEGRATION ANALYSIS

Evaluating practical deployments across the AWS Free Tier highlights distinct technical trade-offs between administrative flexibility and deployment velocity:

- **IaaS Practical Deployment (Amazon EC2):** Launching an Ubuntu Linux instance on Amazon EC2 required setting up Virtual Private Cloud (VPC) subnets, configuring Security Group firewall rules (inbound ports 22 for SSH and 80 for HTTP), attaching Elastic Block Store (EBS) persistent volumes, and manually compiling Apache/Nginx web servers. This offers maximum customizability for legacy software, specialized OS kernels, or complex database tuning, but demands continuous OS patching and security maintenance.
- **PaaS Practical Deployment (AWS Elastic Beanstalk):** Deploying a Node.js/Python application via Elastic Beanstalk reduced setup overhead by 75%. Uploading application source code automatically triggered EC2 provisioning, load balancer wiring, domain DNS mapping, and health monitoring setups. Developers bypass infrastructure assembly while maintaining access to underlying configuration files if needed.
- **SaaS Practical Deployment (AWS Cloud9 / Google Workspace):** Accessing browser-based environments provided immediate functionality without local software installation. Authentication, background updates, database replication, and file storage were completely abstracted, offering instant productivity with zero system administration responsibility.

6. USE CASE SCENARIOS AND DECISION FRAMEWORK

Choosing the correct cloud service model depends on organizational maturity, regulatory compliance requirements, and developer resource availability:

- **Enterprise Infrastructure Migration & Custom Kernels (Select IaaS):** Ideal for migrating legacy enterprise applications, running specialized database engines, or requiring custom network routing and strict operating system kernel configurations.
- **Rapid Web Application Development & Microservices (Select PaaS):** Best suited for startup teams and product developers aiming to bring web APIs, microservices, or mobile backends to market rapidly without maintaining server OS builds or manual scale triggers.

- **Standard Enterprise Productivity & Collaboration Tools (Select SaaS):** Optimal for core organizational functions such as email (Google Workspace/Microsoft 365), customer relationship management (Salesforce), collaborative development (AWS Cloud9/GitHub), and document management.

7. RESULTS

The hands-on exploration across the AWS Free Tier platform confirms that cloud service models exist along a continuum of control versus convenience. IaaS (Amazon EC2) provided complete operational freedom at the expense of setup time and maintenance labor. PaaS (AWS Elastic Beanstalk) reduced deployment lead times by 75% compared to manual EC2 orchestration while providing built-in auto-scaling and monitoring. SaaS solutions eliminated all operational overhead, delivering sub-second deployment velocity. Furthermore, utilizing the AWS Free Tier demonstrated that organizations can design, prototype, and validate multi-tier cloud architectures with zero upfront capital expenditure.

8. CONCLUSION

No single cloud service model fits every enterprise requirement. Modern cloud architectures frequently leverage a hybrid combination across all three paradigms: utilizing SaaS for corporate productivity, PaaS for core customer-facing web services, and IaaS for specialized computational tasks and legacy server hosting. By aligning workload requirements with the appropriate cloud delivery model, organizations achieve maximum engineering agility, optimized resource usage, and robust operational security.